

Cattler

Challenge técnico

El objetivo de esta prueba es evaluar los conocimientos técnicos del candidato en relación con las tareas que deberá afrontar de forma rutinaria en su posición como Backend Developer.

Cattler es una empresa cuyo principal producto es un sistema de trazabilidad para [Feedlots](#). En nuestro modelo de datos, un Feedlot es denominado “Feedyard” y actúa como unidad “tenant” que concentra todos los datos de un usuario típico (lotes, animales, insumos, stock, usuarios, eventos, etc).

Enfocándonos en los niveles de organización del ganado, un Feedyard tiene varios Lotes. Cada Lote a su vez está compuesto por un número de animales, normalmente de similares características (edad, peso, tipo, sexo, etc). Un Lote también puede estar subdividido en varios sublotes, por ejemplo, si los animales que lo componen se encuentran ubicados en dos corrales diferentes, estos animales pertenecerán a dos sublotes diferentes que comparten el mismo Lote base.

Nuestra aplicación principal es un sistema de registro de todas las operaciones que se realizan en el campo (algunas se registran manualmente, otras de forma automática con integraciones de hardware).

Algunas de las operaciones típicas pueden ser: ingreso de animales, registro de aplicación de tratamientos sanitarios, registro del estado de salud de animales particulares, registro de muertes, salidas de animales por venta, etc.

Otras operaciones incluyen: determinar raciones, planificar su entrega a los diferentes corrales, y llevar registro de sobrantes para posteriormente calcular el peso ganado según lo alimentado.

El objetivo es que el encargado de un establecimiento pueda tener una representación digital del campo, enriquecida con informes y detalles difíciles de extraer de un cuaderno de anotaciones físico, y en base a toda la información que Cattler organiza y agrega, poder tomar mejores decisiones sobre la operación del establecimiento ganadero.

El output de esta prueba será evaluado por los desarrolladores actualmente a cargo del backend y en una posterior entrevista se buscará entender el porqué de las decisiones tomadas para resolver el challenge.

Mini API de gestión de lotes de animales

Construir una API REST en Django + DRF + SQLite con autenticación vía TokenAuthentication de DRF que modele:

- Feedyard (tenant). Con nombre.
- User (usuario autenticado) con nombre, feedyard al que pertenece e idioma (EN/ES).
- Lot (lote de animales) perteneciente a un feedyard. Con nombre y feedyard.
- Animal (animal individual) perteneciente a un Lote, con fecha de ingreso, peso inicial y peso actual.
- WeightRecord (pesaje diario del animal), con animal, fecha (sin hora) y peso (float). Debe tener un unique constraint sobre (animal, fecha).

Aislación multi-tenant (requerido)

Un usuario nunca debe poder leer ni modificar datos de un Feedyard distinto al suyo, incluso ante IDs explícitos en el path o el body. Este invariante debe estar enforced a nivel de querysets/permissions y cubierto por tests.

Tareas requeridas

1. Middleware de localización: según el usuario autenticado, activar la localización (idioma) para el ciclo de vida de la request. Los mensajes traducibles deben usar gettext/gettext_lazy, con archivos .po/.mo compilados para EN y ES.
2. Endpoint de creación de Lote: crea un Lote para el Feedyard del usuario, indicando cantidad de animales, fecha de ingreso y peso promedio.
 - Validaciones: cantidad de animales y peso estrictamente mayores a 0; fecha de ingreso no futura.
 - Ante error de validación, devolver HTTP 409 con un mensaje explicativo en el idioma del usuario.
 - Crea los Animals con el peso promedio como peso inicial y actual, y crea un WeightRecord por cada animal por cada día desde la fecha de ingreso hasta hoy (inclusive).
 - La creación de Animals y WeightRecords debe hacerse con operaciones bulk (bulk_create o equivalente).
3. Endpoint de actualización de peso del Lote: setea el peso promedio del lote para una fecha específica.
 - Validaciones: fecha no futura, fecha no anterior al ingreso del lote.
 - Hace upsert del WeightRecord de cada animal para la fecha indicada y para todos los días posteriores hasta hoy, de modo que siempre exista un WeightRecord continuo entre el ingreso y la fecha actual.

- Como "hoy" siempre cae dentro del rango actualizado, el peso_actual de cada animal debe quedar sincronizado con el nuevo valor.
4. Endpoint de ADG: devuelve el aumento promedio diario de peso de un lote en un rango [fecha_inicial, fecha_final].
- Fórmula: $ADG = (\text{peso_final} - \text{peso_inicial}) / \text{días_diferencia}$, donde peso_final y peso_inicial son las sumatorias de los pesos de los animales del lote (vía WeightRecord) en cada fecha.
5. Django Admin para Lot: list_display debe incluir nombre del lote, feedyard, cantidad de animales, fecha de ingreso, peso promedio actual y ADG desde el ingreso hasta hoy.
- La vista debe poder mostrar 1000 lotes sin degradación: resolver los campos calculados con annotate/agregaciones.

Tests (pytest)

Como mínimo:

- Endpoint de creación de Lote: camino feliz y todos los caminos de validación (status code + mensaje traducido).
- Aislación multi-tenant: un usuario del Feedyard A no puede crear/leer/modificar recursos del Feedyard B.

Migración seed

Una migración con datos seed que incluya varios Feedyards con sus Lotes, Animales y los WeightRecord correspondientes.

Entregables

- Repo git con instrucciones para correrlo.
- README.md con decisiones técnicas, trade-offs y supuestos.
- Tests corriendo con pytest.